Exp: 1 **Date:**

Downloading and installing Hadoop; Understanding different Hadoop modes. Start-up scripts, Configuration files

Aim:

To download and install Hadoop and understanding different Hadoop modes. Start-up scripts, Configuration files.

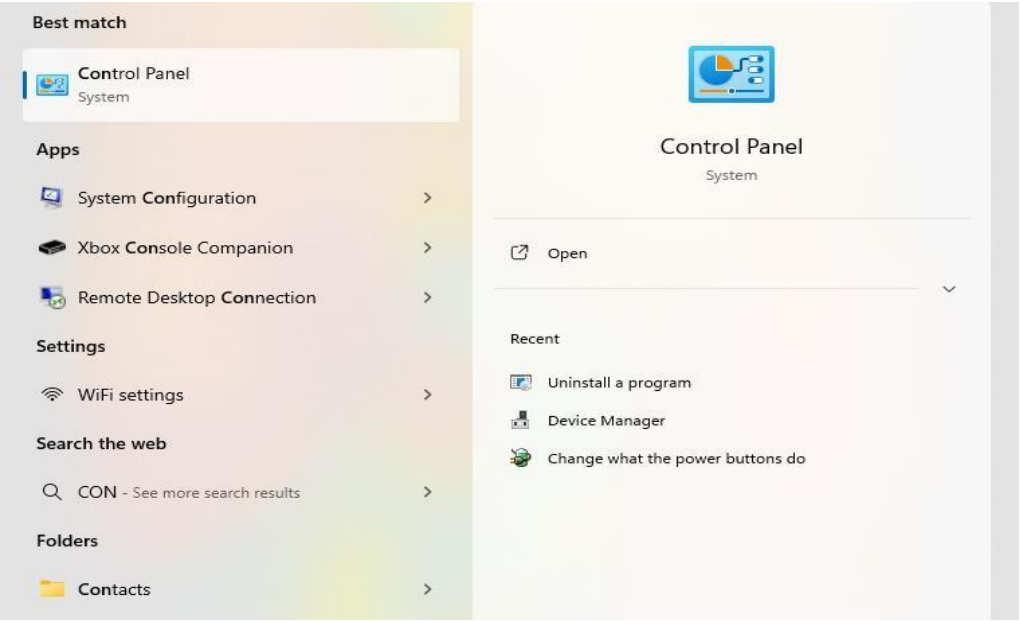
Algorithm:

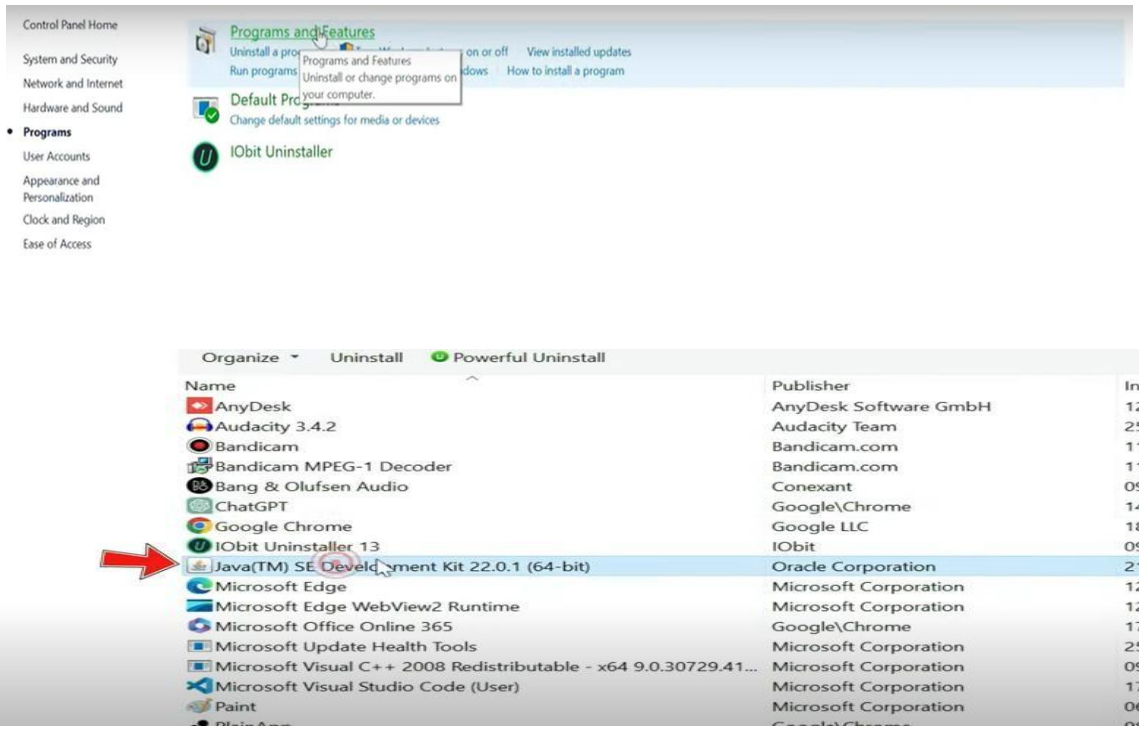
1. Download Hadoop from the official Apache website.
2. Install Java and configure environment variables.
3. Extract Hadoop and set up configuration files (core-site.xml, hdfs-site.xml).
4. Understand and configure different Hadoop modes: local, pseudo-distributed, and fully-distributed.
5. Set up SSH for Hadoop services.
6. Use start-up scripts to launch Hadoop daemons.
7. Verify installation using Hadoop version and web UI.

Procedure:

Step 1: Uninstall Of Existing Java Application

Go To Control Panel → Programs → Programs And Features → Search For Java → Uninstall Java

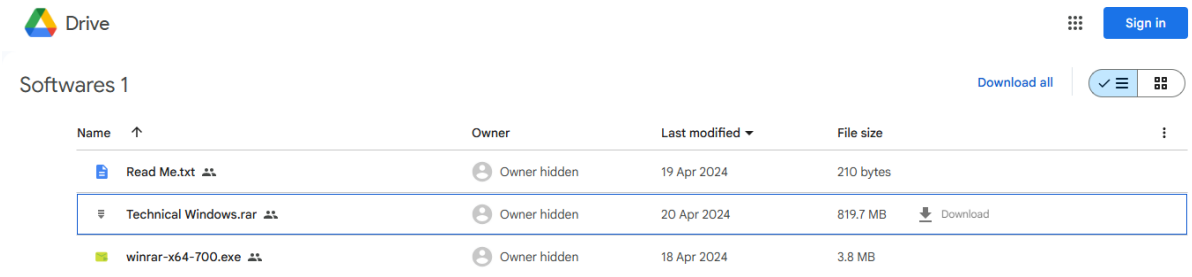






Step 2: Installation Of Hadoop And Java Packages

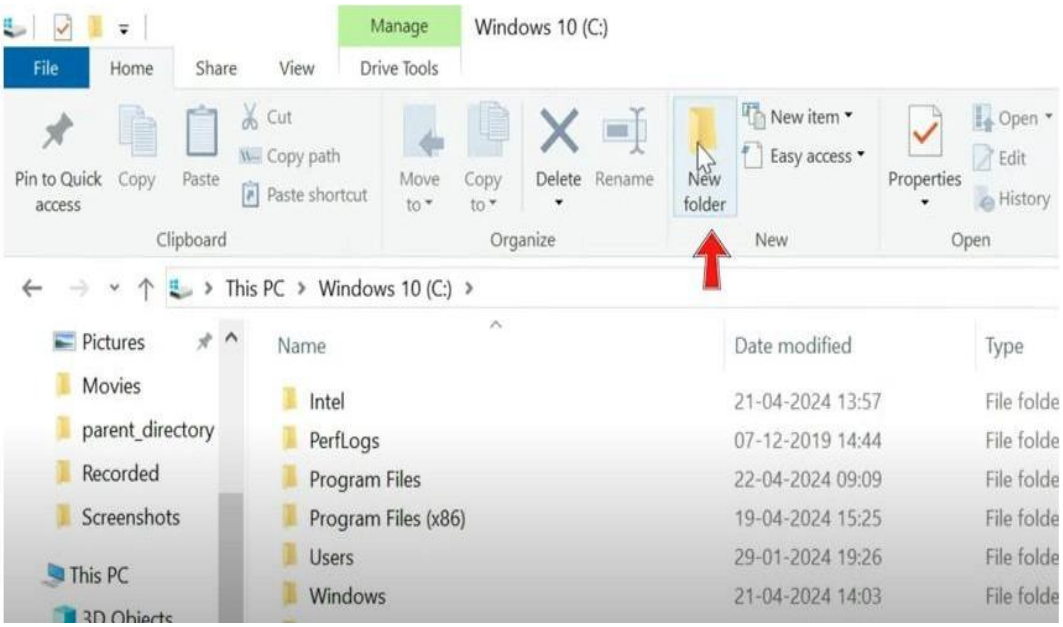
Open Given Link(Https://Shorturl.At/Imedm) → Download 2nd And 3rd Files(Technical Windows And Winrar).

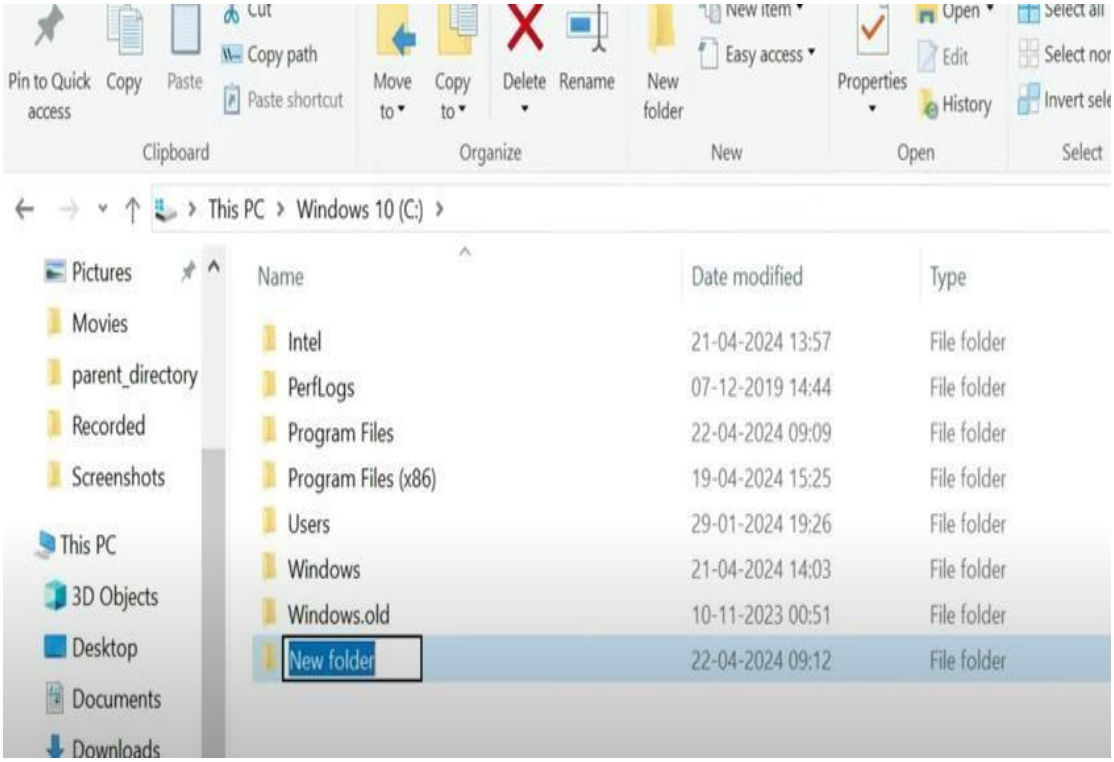


Step 3: Create A New Folder In The Name Of Java

Go To File Explorer → This Pc → Windows 10 (Number

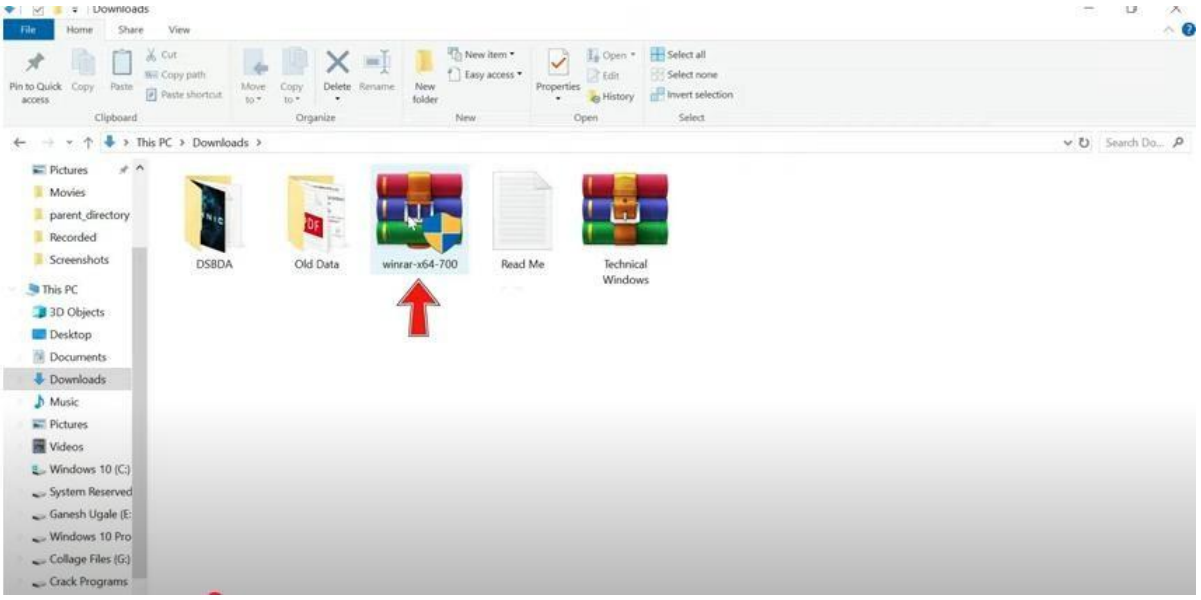
Depends On The Window Version) → Create New Folder → Rename As Java

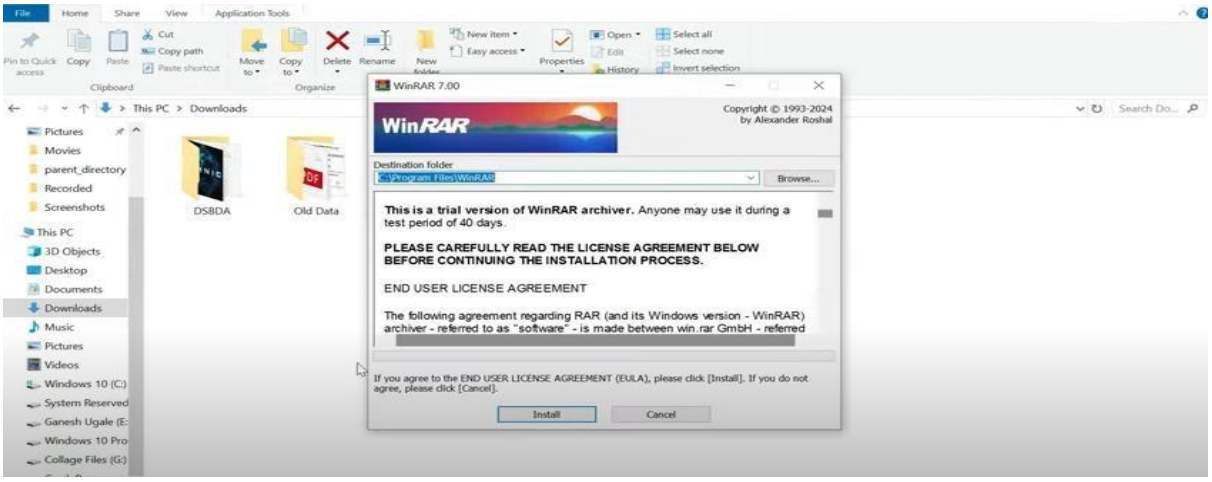




Step 4: Installation Of Winrar

File Explorer → Downloads → Install Winrar → Give All Permissions And Check The Winrar Setup Boxes Which Are Tick In The Below Photo.



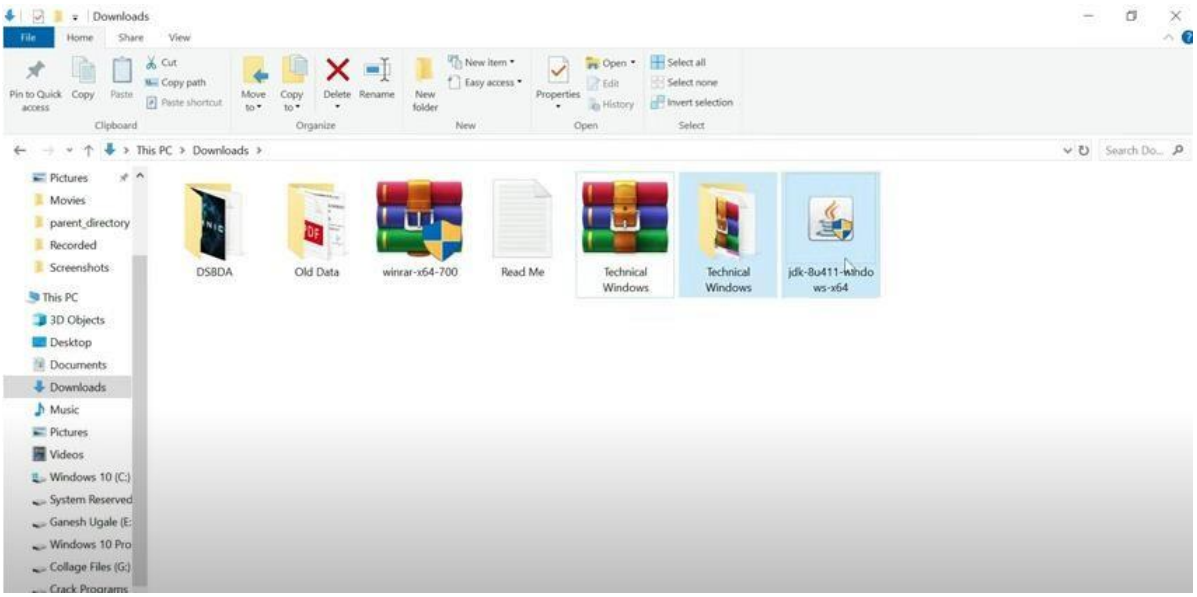


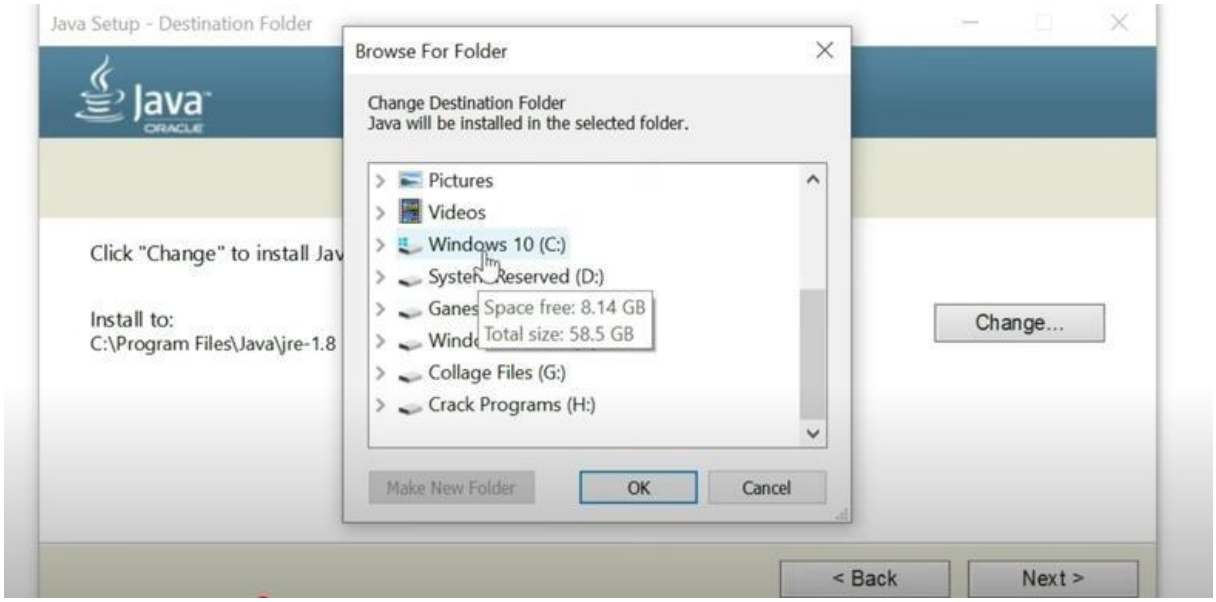
Step 5: Set Up Of Hadoop And Java Files

File Explorer → Downloads → Give Right Click On “Technical Windows File” → Click On “Extract Here” Option → Then Automatically Jdk File Is Extracted →

Click On That “Jdk File” And Give Permissions By Clicking Next After That It Shows Destination Folder And There Is A “Change” Option → Click On That “Change” Option And Change The Destination Folder To Windows 10(C) By Clicking Windows 10 (C) It Shows “Java” Folder Which We Created In Above Step → Select That “Java” → Then

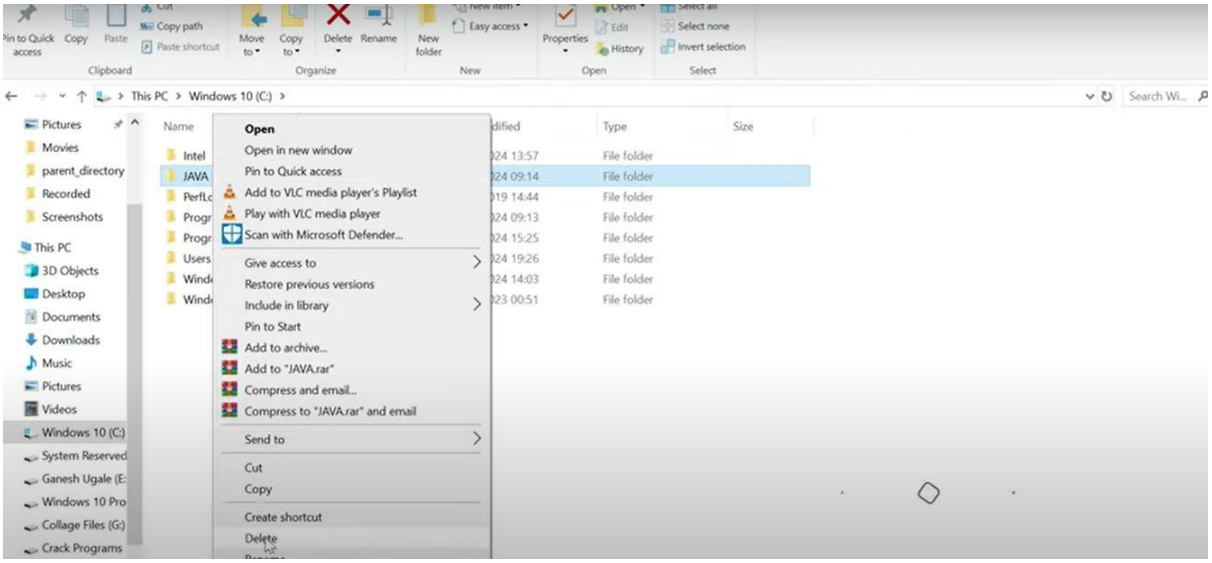
Destination Folder Changes To Java → Then Click Option “Next” So That Jdk Is Installed And Ready To Run.

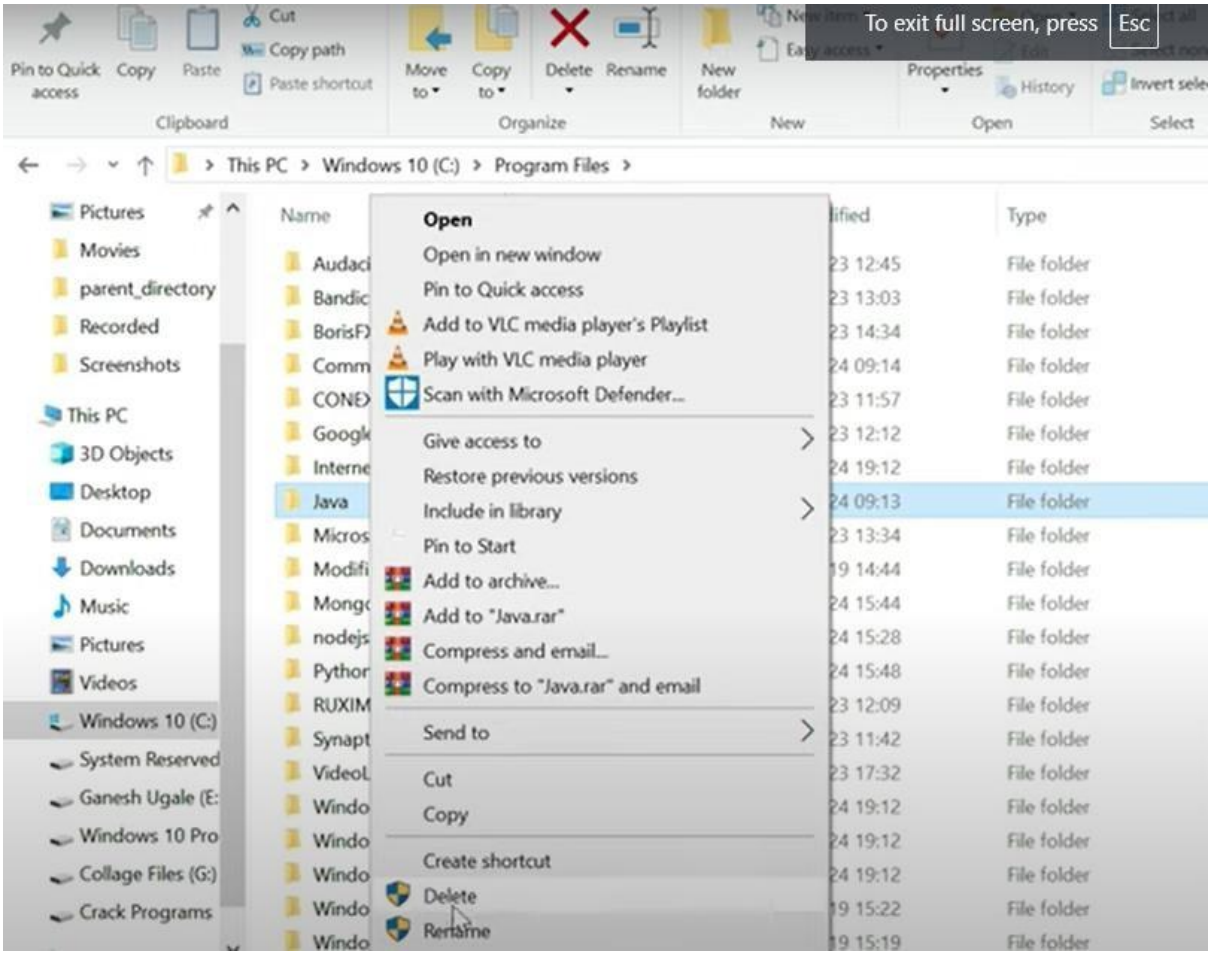

Downloads
File
Home
Share
View
This PC > Downloads >
Pictures
Movies
parent_directory
Recorded
Screenshots
This PC
3D Objects
Desktop
Documents
Downloads
Music
Pictures
Videos
Windows 10 (C:)
System Reserved
Ganesh Ugale (E:
Windows 10 Pro
Collage Files (G:)
DS8DA
Old Data
winrar-x64-700
Read Me
Technical Windows
Technical Windows
jdk-8u411-windows-x64


Java Setup - Destination Folder
Java
ORACLE
Click "Change" to install Jav
Install to:
C:\Program Files\Java\jre-1.8
Change...
Browse For Folder
Change Destination Folder
Java will be installed in the selected folder.
Pictures
Videos
Windows 10 (C:)
Space free: 8.14 GB
Total size: 58.5 GB
Collage Files (G:)
Crack Programs (H:)
Make New Folder
OK
Cancel
< Back
Next >

Step 6: Now Uninstall The Java Folder Which We Created In Above Step

File Explorer → This Pc → Windows 10 → Uninstall Java Folder And As Well As Java Folder In Program Files Which Created Automatically Delete That Java Folder Also.

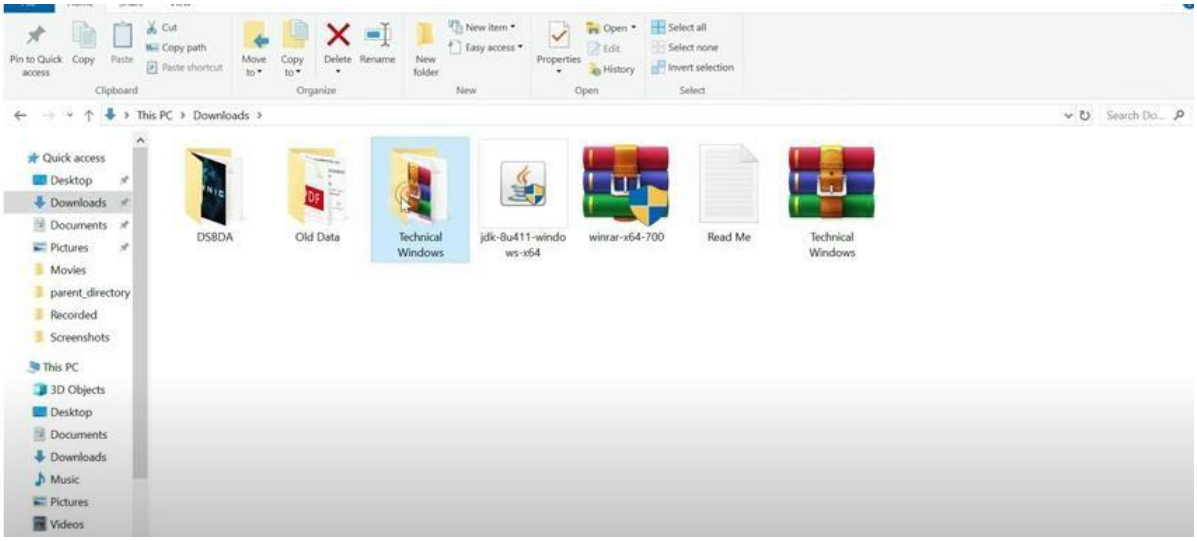




Step 7: Setting Up Files

File Explorer → Downloads → Open “Technical Windows”

→ Copy “Set Up Files” → Paste It In “Windows 10” Directly → After Pasting Setup Files , Right Click On It And Click On The “Extract Here” Option → Then It Automatically Creates Java And Hadoop Folders.



Step 8: Setting Up Environment Path

File Explorer → Windows 10 → Open “Java Folder” And In That Open “Bin” Copy The Java Bin Address (Ie This Pc >

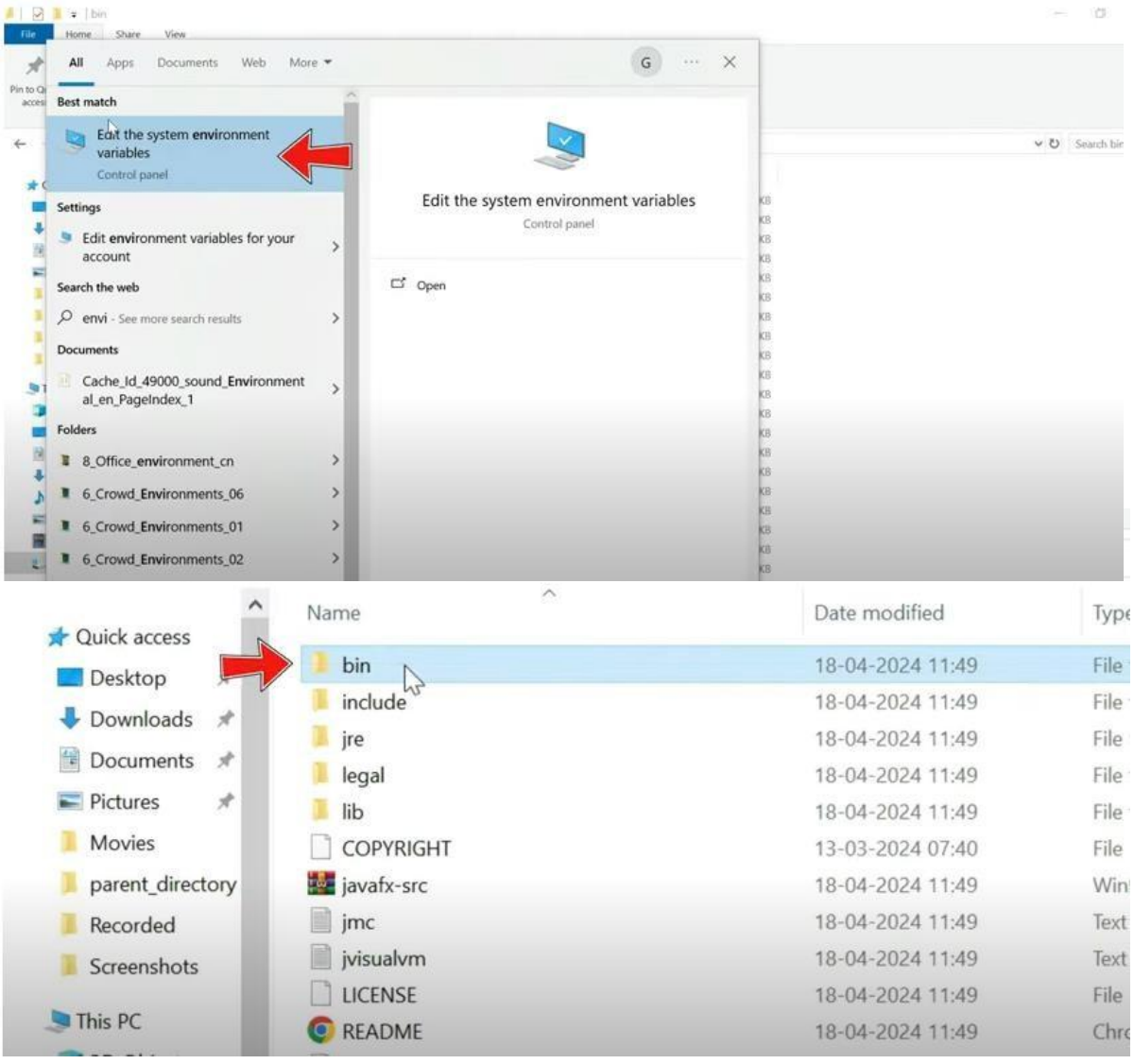
Windows 10(C) > Java > Jdk-1.8 > Bin) → Now Open “ Edit The System Environment Variables” In Apps → Click On The “Environment Variables” Option Which Is At Bottom → Then It Shows Two Boxes (User Variables And Environment Variables) → In “User Variables Box” Click On “New” Option Which Is At The Bottom Of The Box → Then It Shows A New Box With “Variable Name” And “Variable Value” → Enter “Java_Home” In “Variable Name” And

Enter “Java Bin Address”(Ie Which Is Copied Previously) Then Click Ok → Now In “Environment Variables” Select Path And Click “Edit” Option → Then Click On “New” And Add Java Address By Pasting Then Click “Ok” .

Step 9: Now Repeat “Step 7” But Open “Hadoop Folder” → Copy “Hadoop Bin Address” → Open “Edit The System Environment” → Add “Hadoop_Home” And Paste Hadoop Bin Address In “User Variable Section And Add Hadoop Bin Address In Environment Variable As Like As Done In “Step 7”.

Step 10: After Adding Both Java And Hadoop Paths

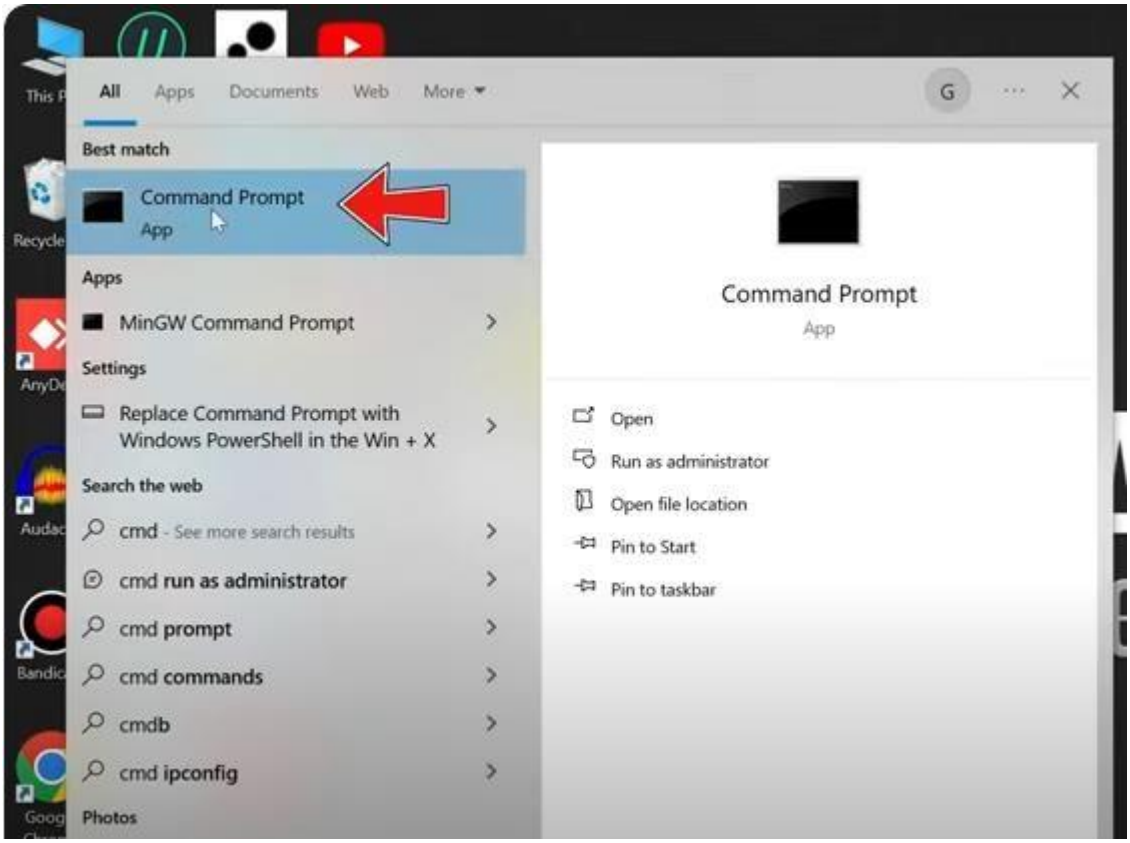
Now Select Path In “Environment Variables” → Click On “Edit” Option And Click On “New” → Add Both “C:\Hadoop\Sbin” And “C:\Hadoop” Addressess In Path Which Is In Environment Variables Section Then Click Ok.



Step 11: Executing Programs In Command Prompt

Open “Command Prompt” → Type “Java –Version” And Enter Then Java Is Running → After Java Is Running Now Type “Hadoop Version” And Enter Then Hadoop Also Running

Successfully → Now Execute Programs.


All
Apps
Documents
Web
More
Best match
Command Prompt
App
Apps
MinGW Command Prompt
Settings
Replace Command Prompt with Windows PowerShell in the Win + X
Search the web
cmd - See more search results
cmd run as administrator
cmd prompt
cmd commands
cmdb
cmd ipconfig
Photos
Command Prompt
App
Open
Run as administrator
Open file location
Pin to Start
Pin to taskbar


Microsoft Windows [Version 10.0.19045.3803]
(c) Microsoft Corporation. All rights reserved.
C:\Users\ganes>java -version
java -version

Result:

Thus downloading and installing Hadoop; Understanding different Hadoop modes. Start-up scripts, Configuration file is successfully executed and output is verified.

Exp: 2 **Date:**

Hadoop Implementation of file management tasks, such as Adding files and directories, Retrieving files and Deleting files

Aim:

To execute hadoop Implementation of file management tasks, such as Adding files and directories, Retrieving files and Deleting files.

Algorithm:

1. Start Hadoop services using start-dfs.sh and start-yarn.sh..
2. Use `hdfs dfs -mkdir` to create directories in HDFS.
3. Use `hdfs dfs -put` to upload files to HDFS.
4. Use `hdfs dfs -ls` to list files and directories.
5. Use `hdfs dfs -cat` or `-get` to retrieve files.
6. Use `hdfs dfs -rm` to delete files from HDFS.
7. Practice file operations to understand HDFS behavior.

Procedure:

1. Create a directory in HDFS at given path(s).

SYNTAX :-

hadoop fs -mkdir <paths>

Example:

hadoop fs -mkdir /user/saurzcode/dir1 /user/saurzcode/dir2

2. List the contents of a directory.

SYNTAX :-

hadoop fs -ls <args>

Example:-

hadoop fs -ls /user/saurzcode

3. Upload and download a file in HDFS.

SYNTAX :-

hadoop fs -put:

Copy single src file, or multiple src files from local file system to the Hadoop data file system

SYNTAX:-

hadoop fs -put <localsrc> ... <HDFS_dest_Path>

Example:

hadoop fs -put /home/saurzcode/Samplefile.txt /user/

saurzcode/dir3/Download:hadoop fs -get:

Copies/Downloads files to the local file system

SYNTAX:-

hadoop fs -get <hdfs_src> <localdst>

Example:

hadoop fs -get /user/saurzcode/dir3/Samplefile.txt /home/

4. See contents of a file

Same as unix cat command:

SYNTAX:-

hadoop fs -cat <path[filename]>

Example:

hadoop fs -cat /user/saurzcode/dir1/abc.txt

5. Copy a file from source to destination

This command allows multiple sources as well in which case the destination must be a directory.

SYNTAX:-

hadoop fs -cp <source> <dest>

Example:

hadoop fs -cp /user/saurzcode/dir1/abc.txt /user/saurzcode/ dir2

6. Copy a file from/To Local file system to HDFS CopyFromLocal

SYNTAX:-

hadoop fs -copyFromLocal <localsrc> URI

Example:

hadoop fs -copyFromLocal /home/saurzcode/abc.txt /user/ saurzcode/abc.txt

Similar to put command, except that the source is restricted to a local file reference.

copyToLocal

SYNTAX:-

hadoop fs -copyToLocal [-ignorecrc] [-crc] URI <localdst>

Similar to get command, except that the destination is restricted to a local file reference.

7. Move file from source to destination.

Note:- Moving files across filesystem is not permitted.

SYNTAX:-

hadoop fs -mv <src> <dest>

Example:

hadoop fs -mv /user/saurzcode/dir1/abc.txt /user/saurzcode/ dir2

8. Remove a file or directory in HDFS.

Remove files specified as argument. Deletes directory only when it is empty

SYNTAX:-

hadoop fs -rm <arg>

Example:

hadoop fs -rm /user/saurzcode/dir1/abc.txt Recursive version of delete.

SYNTAX:-

hadoop fs -rmr <arg>

Example:

hadoop fs -rmr /user/saurzcode/

9. Display last few lines of a file.Similar to tail command in Unix.

SYNTAX:-

hadoop fs -tail <path[filename]>

Example:

hadoop fs -tail /user/saurzcode/dir1/abc.txt

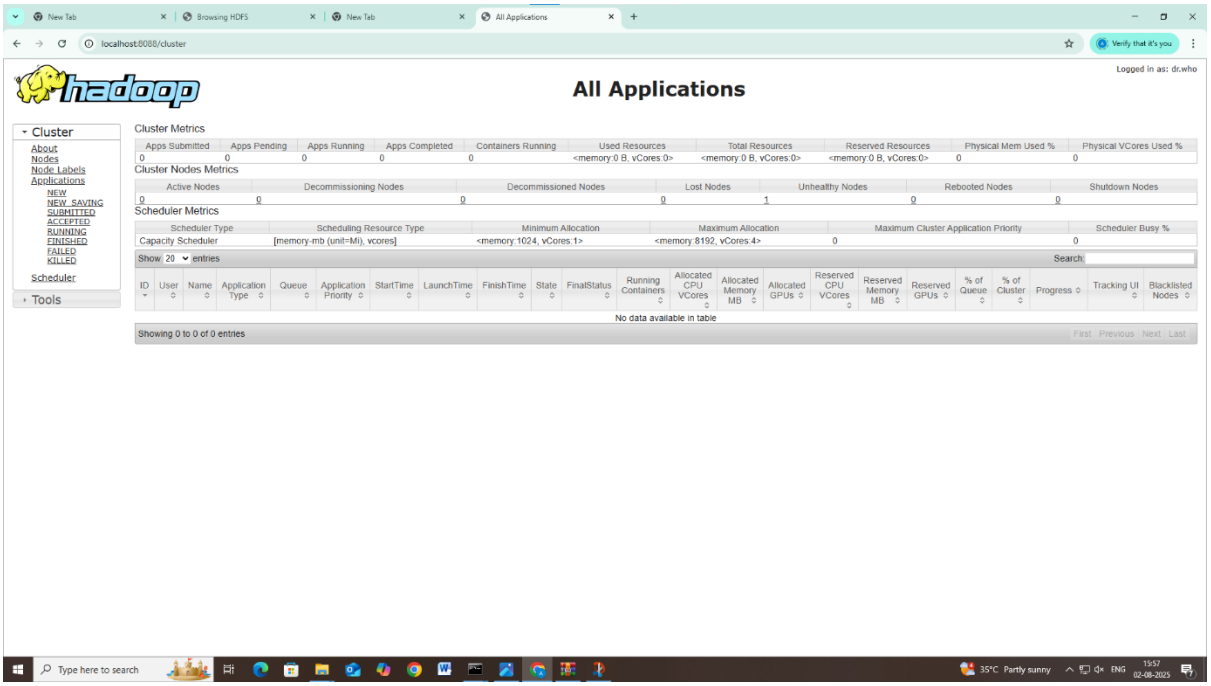
10. Display the aggregate length of a file.

SYNTAX:-

hadoop fs-du <path>

Example:

hadoop fs -du /user/saurzcode/dir1/abc.txt.



Result:

Thus the hadoop Implementation of file management tasks, such as Adding files and directories, Retrieving files and Deleting files is successfully executed and output is verified.

Exp: 3 **Date:**

Implement of Matrix Multiplication with Hadoop Map Reduce

Aim:

To write a program to implement Matrix Multiplication with Hadoop Map Reduce.

Algorithm:

1. Prepare input matrices in key-value format.
2. Write a Mapper to emit intermediate matrix elements.
3. Write a Reducer to compute the product of matching elements.
4. Configure job to handle multiple inputs.
5. Compile and run the MapReduce job.
6. Store output in HDFS and verify results.
7. Analyze performance and scalability of the solution.

Program:

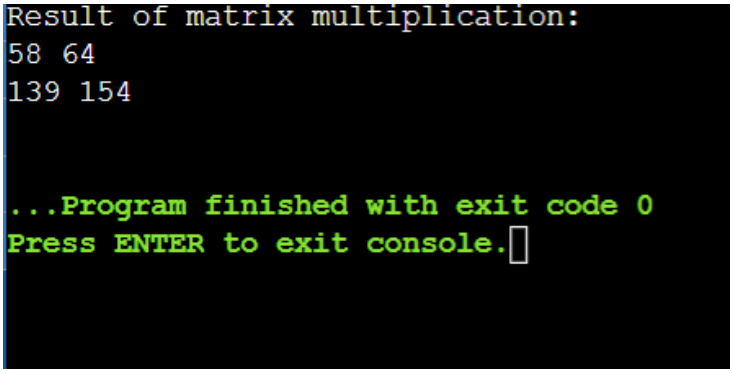
```
public class Main {
   public static void main(String[] args) {
      int[][] A = {
         {1, 2, 3},
         {4, 5, 6}
      };
      int[][] B = {
         {7, 8},
         {9, 10},
         {11, 12}
      };
      int rowsA = A.length;
      int colsA = A[0].length;
      int colsB = B[0].length;
      int[][] result = new int[rowsA][colsB];
```

```
        // Multiply A and B



        for (int i = 0; i < rowsA; i++) {
            for (int j = 0; j < colsB; j++) {
                int sum = 0;
                for (int k = 0; k < colsA; k++) {
                    sum += A[i][k] * B[k][j];
                }
                result[i][j] = sum;
            }
        }
        // Print the result
        System.out.println("Result of matrix multiplication:");
        for (int i = 0; i < rowsA; i++) {
            for (int j = 0; j < colsB; j++) {
                System.out.print(result[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

Output:



Result:

Thus the program to implement Matrix Multiplication with Hadoop Map Reduce is successful and output is verified.

Exp: 4 **Date:**

Run a basic Word Count Map Reduce program to understand Map Reduce Paradigm

Aim:

To implement Matrix Multiplication with Hadoop Map Reduce.

Algorithm:

1. Prepare a text file as input in HDFS.
2. Write a Mapper to tokenize words and emit (word, 1).
3. Write a Reducer to sum counts for each word.
4. Compile and package the MapReduce program.
5. Run the job using Hadoop command line.
6. View output file with word frequencies.
7. Understand MapReduce flow through this example.

Program:

```
import java.util.*;

public class Main {

   // Mapper: takes a line and emits (word, 1)

   public static List<Map.Entry<String, Integer>> map(String line) {

      List<Map.Entry<String, Integer>> results = new ArrayList<>();

      String[] words = line.toLowerCase().replaceAll("[^a-z ]", "").split("\\s+");

      for (String word : words) {

         if (!word.isEmpty()) {

            results.add(new AbstractMap.SimpleEntry<>(word, 1));

         }

      }

      return results;

   }

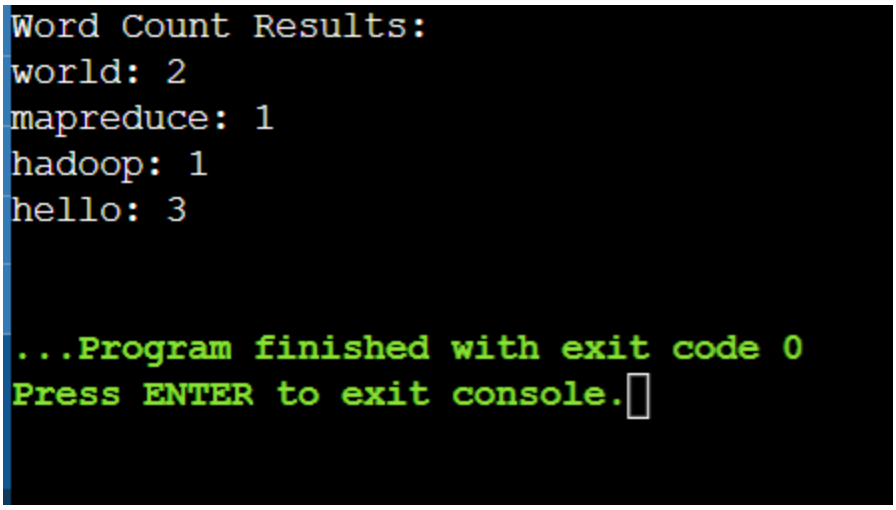

   // Shuffle & Sort: groups mapped results by key

   public static Map<String, List<Integer>> shuffle(List<Map.Entry<String, Integer>> mappedData) {
```

```
Map<String, List<Integer>> grouped = new HashMap<>();
        for (Map.Entry<String, Integer> entry : mappedData) {
            grouped.computeIfAbsent(entry.getKey(), k -> new ArrayList<>()).add(entry.getValue());
        }
        return grouped;
    }
    // Reducer: sums all counts for a key
    public static Map<String, Integer> reduce(Map<String, List<Integer>> groupedData) {
        Map<String, Integer> reduced = new HashMap<>();
        for (Map.Entry<String, List<Integer>> entry : groupedData.entrySet()) {
            int sum = 0;
            for (int count : entry.getValue()) {
                sum += count;
            }
            reduced.put(entry.getKey(), sum);
        }
        return reduced;
    }
    public static void main(String[] args) {
        // Example input lines
        String[] lines = {
            "Hello world",
            "Hello Hadoop",
            "Hello MapReduce world"
        };
        // Map phase
        List<Map.Entry<String, Integer>> mapped = new ArrayList<>();
        for (String line : lines) {
            mapped.addAll(map(line));
```

```
        }
        // Shuffle phase
        Map<String, List<Integer>> grouped = shuffle(mapped);
        // Reduce phase
        Map<String, Integer> reduced = reduce(grouped);
        // Output result
        System.out.println("Word Count Results:");
        for (Map.Entry<String, Integer> entry : reduced.entrySet()) {
            System.out.println(entry.getKey() + ": " + entry.getValue());
        }
    }
}
```

Output:



Result:

Thus the program to run a basic Word Count Map Reduce program to understand Map Reduce Paradigm.

Exp: 5 **Date:**

Implementation of K-means clustering using Map Reduce

Aim:

To write a program to implement K-means clustering using Map Reduce.

Algorithm:

1. Initialize centroids and prepare input data.
2. Write a Mapper to assign points to nearest centroid.
3. Write a Reducer to compute new centroids.
4. Iterate MapReduce job until centroids stabilize.
5. Store final clusters in HDFS.
6. Visualize or interpret clustering results.
7. Evaluate clustering quality and convergence.

Program:

```
import java.util.*;

public class Main {

   static class Point {

      float[] coords;

      Point(String[] values) {

         coords = new float[values.length];

         for (int i = 0; i < values.length; i++)

            coords[i] = Float.parseFloat(values[i]);

      }

      float distance(Point other, int p) {

         float sum = 0;

         for (int i = 0; i < coords.length; i++) {

            float diff = Math.abs(coords[i] - other.coords[i]);

            sum += (p == 1) ? diff : diff * diff;

         }

         return (p == 1) ? sum : (float) Math.sqrt(sum);
```

```
        }
        public String toString() { return Arrays.toString(coords); }
    }
    public static void main(String[] args) {
        int k = 2, p = 2;
        String[][] centroidStrings = { {"0.0", "0.0"}, {"5.0", "5.0"} };
        Point[] centroids = new Point[k];
        for (int i = 0; i < k; i++) centroids[i] = new Point(centroidStrings[i]);
        Point point = new Point("2.0,2.0".split(","));
        float minDist = Float.MAX_VALUE;
        int nearest = -1;
        for (int i = 0; i < k; i++) {
            float d = point.distance(centroids[i], p);
            if (d < minDist) { minDist = d; nearest = i; }
        }
        System.out.println("Mapper Output:");
        System.out.println("Centroid Index: " + nearest);
        System.out.println("Point: " + point);
    }
}
```

Output:

```
Mapper Output:
Centroid Index: 0
Point: [2.0, 2.0]


...Program finished with exit code 0
Press ENTER to exit console.
```

Result:

Thus the program to write for implementing K-means clustering using Map Reduce.

Exp: 6 **Date:**

Implement and demonstrate the FIND-S Algorithm for finding the most specific hypothesis based on a given set of training data samples. Read the training data from a CSV file

Aim:

To write a program to implement and demonstrate the FIND-S Algorithm for finding the most specific hypothesis based on a given set of training data samples read the training data from a CSV file.

Algorithm:

1. Load training data from a CSV file.
2. Initialize the most specific hypothesis.
3. For each positive example, generalize hypothesis.
4. Ignore negative examples in FIND-S.
5. Update hypothesis only when necessary.
6. Output final specific hypothesis.
7. Verify consistency with training data.

Program:

```
import java.util.*;

public class Main {

   static class Point {

      double x, y;

      Point(double x, double y) { this.x = x; this.y = y; }

      double distance(Point o) { return Math.hypot(x - o.x, y - o.y); }

      Point add(Point o) { return new Point(x + o.x, y + o.y); }

      Point divide(int d) { return new Point(x / d, y / d); }

      public String toString() { return String.format("(%.1f, %.1f)", x, y); }

   }

   static Map<Integer, List<Point>> map(List<Point> points, List<Point> centroids) {

      Map<Integer, List<Point>> clusters = new HashMap<>();

      for (int i = 0; i < centroids.size(); i++) clusters.put(i, new ArrayList<>());

      for (Point p : points) {

         int idx = 0; double min = Double.MAX_VALUE;
```

```
        for (int i = 0; i < centroids.size(); i++) {

            double dist = p.distance(centroids.get(i));
            if (dist < min) { min = dist; idx = i; }
        }
        clusters.get(idx).add(p);
    }
    return clusters;
}
static List<Point> reduce(Map<Integer, List<Point>> clusters) {
    List<Point> newCentroids = new ArrayList<>();
    for (List<Point> group : clusters.values()) {
        if (group.isEmpty()) { newCentroids.add(new Point(0, 0)); continue; }
        Point sum = new Point(0, 0);
        for (Point p : group) sum = sum.add(p);
        newCentroids.add(sum.divide(group.size()));
    }
    return newCentroids;
}
public static void main(String[] args) {
    List<Point> data = Arrays.asList(
        new Point(1,2), new Point(1.5,1.8), new Point(5,8), new Point(8,8),
        new Point(1,0.6), new Point(9,11), new Point(8,2), new Point(10,2), new Point(9,3)
    );
    int k = 3, iterations = 5;
    List<Point> centroids = new ArrayList<>(data.subList(0, k));
    System.out.println("Initial Centroids: " + centroids);
    for (int i = 1; i <= iterations; i++) {
        System.out.println("\nIteration " + i);
        Map<Integer, List<Point>> clusters = map(data, centroids);
```

```
        for (int j = 0; j < k; j++)
            System.out.println("Cluster " + j + ": " + clusters.get(j));
        centroids = reduce(clusters);
        System.out.println("Updated Centroids: " + centroids);
    }
    System.out.println("\nFinal Centroids:");
    for (int i = 0; i < centroids.size(); i++)
        System.out.println("Cluster " + i + ": " + centroids.get(i));
  }
}
```

Output :

```
Cluster 1: []
Cluster 2: [(5.0, 8.0), (8.0, 8.0), (9.0, 11.0), (8.0, 2.0), (10.0, 2.0), (9.0, 3.0)]
Updated Centroids: [(1.2, 1.5), (0.0, 0.0), (8.2, 5.7)]

Iteration 3
Cluster 0: [(1.0, 2.0), (1.5, 1.8), (1.0, 0.6)]
Cluster 1: []
Cluster 2: [(5.0, 8.0), (8.0, 8.0), (9.0, 11.0), (8.0, 2.0), (10.0, 2.0), (9.0, 3.0)]
Updated Centroids: [(1.2, 1.5), (0.0, 0.0), (8.2, 5.7)]

Iteration 4
Cluster 0: [(1.0, 2.0), (1.5, 1.8), (1.0, 0.6)]
Cluster 1: []
Cluster 2: [(5.0, 8.0), (8.0, 8.0), (9.0, 11.0), (8.0, 2.0), (10.0, 2.0), (9.0, 3.0)]
Updated Centroids: [(1.2, 1.5), (0.0, 0.0), (8.2, 5.7)]

Iteration 5
Cluster 0: [(1.0, 2.0), (1.5, 1.8), (1.0, 0.6)]
Cluster 1: []
Cluster 2: [(5.0, 8.0), (8.0, 8.0), (9.0, 11.0), (8.0, 2.0), (10.0, 2.0), (9.0, 3.0)]
Updated Centroids: [(1.2, 1.5), (0.0, 0.0), (8.2, 5.7)]

Final Centroids:
Cluster 0: (1.2, 1.5)
Cluster 1: (0.0, 0.0)
Cluster 2: (8.2, 5.7)


...Program finished with exit code 0
Press ENTER to exit console.
```

Result:

Thus a program to implement and demonstrate the FIND-S Algorithm for finding the most specific hypothesis based on a given set of training data samples and read the training data from a CSV file is successfully executed and output is verified.

Exp: 7 **Date:**

For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm to output a description of the set of all hypotheses consistent with the training examples

Aim:

To write a program for a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm to output a description of the set of all hypotheses consistent with the training examples.

Algorithm:

1. Load training data from a CSV file.
2. Initialize specific boundary S and general boundary G.
3. For each example, check if it's positive or negative.
4. Update S and G based on consistency.
5. Remove inconsistent hypotheses from both boundaries.
6. Repeat for all examples in the dataset.
7. Output final version space with S and G.

Program:

```
import java.util.*;

public class Main {

   public static void main(String[] args) {

      // Sample dataset: attributes + class label (Y/N)

      // Each example is a String array, last element is class label "Y" or "N"

      List<String[]> dataset = Arrays.asList(

         new String[]{"Sunny", "Warm", "Normal", "Strong", "Warm", "Same", "Y"},

         new String[]{"Sunny", "Warm", "High", "Strong", "Warm", "Same", "Y"},

         new String[]{"Rain", "Cold", "High", "Strong", "Warm", "Change", "N"},

         new String[]{"Sunny", "Warm", "High", "Strong", "Cool", "Change", "Y"}

      );

      // Separate concepts and target

      List<String[]> concepts = new ArrayList<>();

      List<String> target = new ArrayList<>();
```

```
    for (String[] example : dataset) {

        concepts.add(Arrays.copyOfRange(example, 0, example.length - 1));
        target.add(example[example.length - 1]);
    }
    CandidateElimination ce = new CandidateElimination(concepts, target);
    ce.run();
}
static class CandidateElimination {
    List<String[]> concepts;
    List<String> target;
    String[] specific_h;
    List<String[]> general_h;
    CandidateElimination(List<String[]> concepts, List<String> target) {
        this.concepts = concepts;
        this.target = target;
    }
    void run() {
        int numAttributes = concepts.get(0).length;
        // Initialize specific_h to first positive example
        specific_h = null;
        for (int i = 0; i < target.size(); i++) {
            if (target.get(i).equalsIgnoreCase("Y")) {
                specific_h = concepts.get(i).clone();
                break;
            }
        }
        if (specific_h == null) {
            System.out.println("No positive example found.");
            return;
```

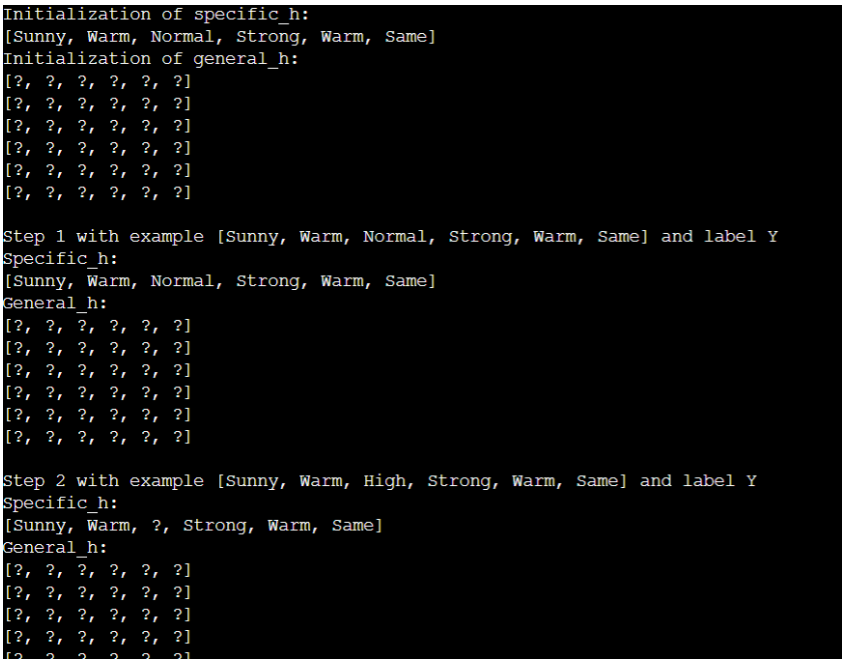
```
}
// Initialize general_h to most general hypotheses

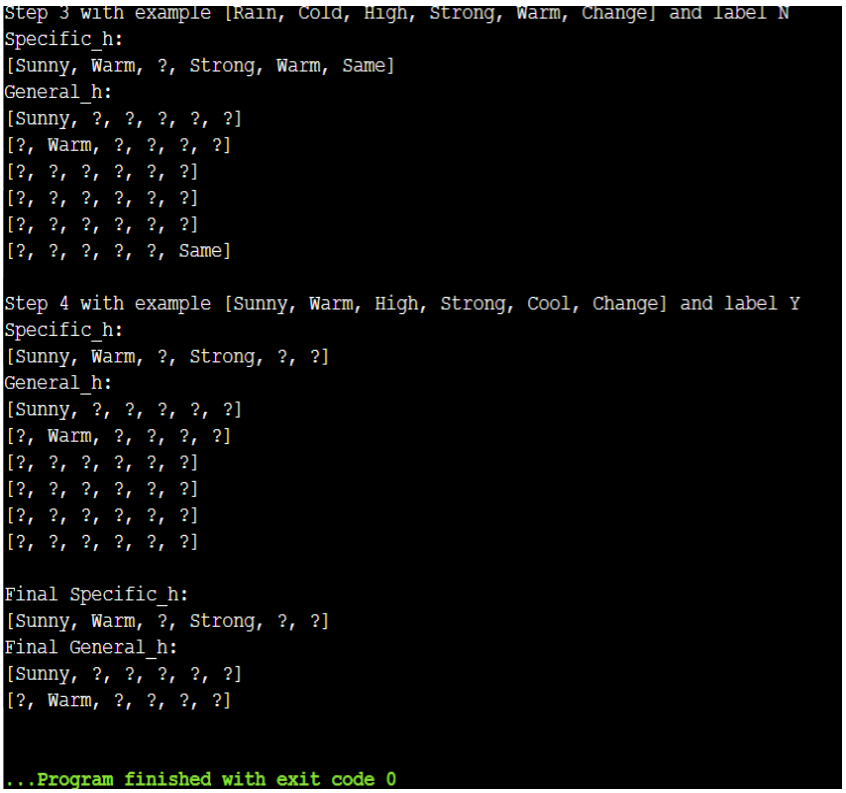
general_h = new ArrayList<>();
for (int i = 0; i < numAttributes; i++) {
    String[] genHypothesis = new String[numAttributes];
    Arrays.fill(genHypothesis, "?");
    general_h.add(genHypothesis);
}
System.out.println("Initialization of specific_h:");
System.out.println(Arrays.toString(specific_h));
System.out.println("Initialization of general_h:");
printGeneralH();
// Process each example
for (int i = 0; i < concepts.size(); i++) {
    String[] instance = concepts.get(i);
    String label = target.get(i);
    if (label.equalsIgnoreCase("Y")) {
        // For positive example, generalize specific_h
        for (int attr = 0; attr < numAttributes; attr++) {
            if (!specific_h[attr].equals(instance[attr])) {
                specific_h[attr] = "?";
                general_h.get(attr)[attr] = "?";
            }
        }
    } else if (label.equalsIgnoreCase("N")) {
        // For negative example, specialize general_h
        for (int attr = 0; attr < numAttributes; attr++) {
            if (!specific_h[attr].equals(instance[attr])) {
                general_h.get(attr)[attr] = specific_h[attr];
```

```
                } else {
                    general_h.get(attr)[attr] = "?";
                }
            }
        }
        System.out.println("\nStep " + (i + 1) + " with example " +
                Arrays.toString(instance) + " and label " + label);
        System.out.println("Specific_h:");
        System.out.println(Arrays.toString(specific_h));
        System.out.println("General_h:");
        printGeneralH();
    }
    // Remove all general hypotheses that are all '?'
    general_h.removeIf(h -> {
        for (String s : h) {
            if (!s.equals("?")) return false;
        }
        return true;
    });
    System.out.println("\nFinal Specific_h:");
    System.out.println(Arrays.toString(specific_h));
    System.out.println("Final General_h:");
    for (String[] h : general_h) {
        System.out.println(Arrays.toString(h));
    }
}
void printGeneralH() {
    for (String[] h : general_h) {
        System.out.println(Arrays.toString(h));
    }
```

```
        }
    }
}
```

Output:


Initialization of specific_h:
[Sunny, Warm, Normal, Strong, Warm, Same]
Initialization of general_h:
[?, ?, ?, ?, ?, ?]
[?, ?, ?, ?, ?, ?]
[?, ?, ?, ?, ?, ?]
[?, ?, ?, ?, ?, ?]
[?, ?, ?, ?, ?, ?]
[?, ?, ?, ?, ?, ?]
Step 1 with example [Sunny, Warm, Normal, Strong, Warm, Same] and label Y
Specific_h:
[Sunny, Warm, Normal, Strong, Warm, Same]
General_h:
[?, ?, ?, ?, ?, ?]
[?, ?, ?, ?, ?, ?]
[?, ?, ?, ?, ?, ?]
[?, ?, ?, ?, ?, ?]
[?, ?, ?, ?, ?, ?]
[?, ?, ?, ?, ?, ?]
Step 2 with example [Sunny, Warm, High, Strong, Warm, Same] and label Y
Specific_h:
[Sunny, Warm, ?, Strong, Warm, Same]
General_h:
[?, ?, ?, ?, ?, ?]
[?, ?, ?, ?, ?, ?]
[?, ?, ?, ?, ?, ?]
[?, ?, ?, ?, ?, ?]


Step 3 with example [Rain, Cold, High, Strong, Warm, Change] and label N
Specific_h:
[Sunny, Warm, ?, Strong, Warm, Same]
General_h:
[Sunny, ?, ?, ?, ?, ?]
[?, Warm, ?, ?, ?, ?]
[?, ?, ?, ?, ?, ?]
[?, ?, ?, ?, ?, ?]
[?, ?, ?, ?, ?, ?]
[?, ?, ?, ?, ?, Same]
Step 4 with example [Sunny, Warm, High, Strong, Cool, Change] and label Y
Specific_h:
[Sunny, Warm, ?, Strong, ?, ?]
General_h:
[Sunny, ?, ?, ?, ?, ?]
[?, Warm, ?, ?, ?, ?]
[?, ?, ?, ?, ?, ?]
[?, ?, ?, ?, ?, ?]
[?, ?, ?, ?, ?, ?]
[?, ?, ?, ?, ?, ?]
Final Specific_h:
[Sunny, Warm, ?, Strong, ?, ?]
Final General_h:
[Sunny, ?, ?, ?, ?, ?]
[?, Warm, ?, ?, ?, ?]
...Program finished with exit code 0

Result:

Thus for a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm to output a description of the set of all hypotheses consistent with the training examples is successfully executed and output is verified.

Exp: 8 **Date:**

Write a program to demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample

Aim:

To write a program to demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

Algorithm:

1. Load dataset from a CSV file.
2. Calculate entropy of the dataset.
3. Compute information gain for each attribute.
4. Select attribute with highest gain to split data.
5. Recursively build tree for each branch.
6. Stop when all samples are classified or no attributes remain.
7. Use the tree to classify new samples.

Program:

```
import java.util.*;

public class Main {

   public static void main(String[] args) {

      // Simulate CSV data with header + rows

      List<String[]> csvData = new ArrayList<>();

      csvData.add(new String[]{"Outlook", "Temperature", "Humidity", "Wind", "PlayTennis"});

      csvData.add(new String[]{"Sunny", "Hot", "High", "Weak", "No"});

      csvData.add(new String[]{"Sunny", "Hot", "High", "Strong", "No"});

      csvData.add(new String[]{"Overcast", "Hot", "High", "Weak", "Yes"});

      csvData.add(new String[]{"Rain", "Mild", "High", "Weak", "Yes"});

      csvData.add(new String[]{"Rain", "Cool", "Normal", "Weak", "Yes"});

      csvData.add(new String[]{"Rain", "Cool", "Normal", "Strong", "No"});

      csvData.add(new String[]{"Overcast", "Cool", "Normal", "Strong", "Yes"});

      csvData.add(new String[]{"Sunny", "Mild", "High", "Weak", "No"});

      csvData.add(new String[]{"Sunny", "Cool", "Normal", "Weak", "Yes"});
```

```
csvData.add(new String[]{"Rain", "Mild", "Normal", "Weak", "Yes"});

csvData.add(new String[]{"Sunny", "Mild", "Normal", "Strong", "Yes"});
csvData.add(new String[]{"Overcast", "Mild", "High", "Strong", "Yes"});
csvData.add(new String[]{"Overcast", "Hot", "Normal", "Weak", "Yes"});
csvData.add(new String[]{"Rain", "Mild", "High", "Strong", "No"});

// Build header maps
Map<String, Integer> nameToIdx = new HashMap<>();
Map<Integer, String> idxToName = new HashMap<>();

String[] header = csvData.get(0);
for (int i = 0; i < header.length; i++) {
    nameToIdx.put(header[i], i);
    idxToName.put(i, header[i]);
}

// Extract data rows (skip header)
List<String[]> dataRows = csvData.subList(1, csvData.size());
System.out.println("Headers: " + Arrays.toString(header));

// Project columns: only keep "Outlook", "Humidity", "PlayTennis"
List<String> columnsToProject = Arrays.asList("Outlook", "Humidity", "PlayTennis");
Data projectedData = projectColumns(header, dataRows, nameToIdx, columnsToProject);

System.out.println("\nProjected Headers: " + Arrays.toString(projectedData.header));
System.out.println("Projected Rows:");
for (String[] row : projectedData.rows) {
    System.out.println(Arrays.toString(row));
}
```

```
        // Get unique values per attribute
        Map<String, Set<String>> uniqueValues = getUniqueValues(projectedData);

        System.out.println("\nUnique values per attribute:");
        for (String attr : uniqueValues.keySet()) {
            System.out.println(attr + ": " + uniqueValues.get(attr));
        }
    }

    // Container class for projected data
    static class Data {
        String[] header;
        List<String[]> rows;
        Map<String, Integer> nameToIdx;
        Map<Integer, String> idxToName;

        Data(String[] header, List<String[]> rows, Map<String, Integer> nameToIdx, Map<Integer, String>
idxToName) {
            this.header = header;
            this.rows = rows;
            this.nameToIdx = nameToIdx;
            this.idxToName = idxToName;
        }
    }

    static Data projectColumns(String[] originalHeader, List<String[]> originalRows,
                               Map<String, Integer> originalNameToIdx,
                               List<String> columnsToProject) {
        // Find indices of columns to project
```

```
List<Integer> colIndices = new ArrayList<>();
for (String col : columnsToProject) {
    if (originalNameToIdx.containsKey(col)) {
        colIndices.add(originalNameToIdx.get(col));
    }
}
// Build new header
String[] newHeader = new String[colIndices.size()];
for (int i = 0; i < colIndices.size(); i++) {
    newHeader[i] = originalHeader[colIndices.get(i)];
}
// Project rows
List<String[]> newRows = new ArrayList<>();
for (String[] row : originalRows) {
    String[] newRow = new String[colIndices.size()];
    for (int i = 0; i < colIndices.size(); i++) {
        newRow[i] = row[colIndices.get(i)];
    }
    newRows.add(newRow);
}

// Build new maps
Map<String, Integer> newNameToIdx = new HashMap<>();
Map<Integer, String> newIdxToName = new HashMap<>();
for (int i = 0; i < newHeader.length; i++) {
    newNameToIdx.put(newHeader[i], i);
    newIdxToName.put(i, newHeader[i]);
}

return new Data(newHeader, newRows, newNameToIdx, newIdxToName);
```

```
    }

    static Map<String, Set<String>> getUniqueValues(Data data) {
        Map<String, Set<String>> valMap = new HashMap<>();
        for (String colName : data.nameToIdx.keySet()) {
            valMap.put(colName, new HashSet<>());
        }
        for (String[] row : data.rows) {
            for (int i = 0; i < data.header.length; i++) {
                String colName = data.header[i];
                valMap.get(colName).add(row[i]);
            }
        }
        return valMap;
    }
}
```

Output:

```
Headers: [Outlook, Temperature, Humidity, Wind, PlayTennis]

Projected Headers: [Outlook, Humidity, PlayTennis]
Projected Rows:
[Sunny, High, No]
[Sunny, High, No]
[Overcast, High, Yes]
[Rain, High, Yes]
[Rain, Normal, Yes]
[Rain, Normal, No]
[Overcast, Normal, Yes]
[Sunny, High, No]
[Sunny, Normal, Yes]
[Rain, Normal, Yes]
[Sunny, Normal, Yes]
[Overcast, High, Yes]
[Overcast, Normal, Yes]
[Rain, High, No]

Unique values per attribute:
Outlook: [Rain, Overcast, Sunny]
Humidity: [High, Normal]
PlayTennis: [No, Yes]

=== Code Execution Successful ===
```

Result:

Thus a program to demonstrate the working of the decision tree based ID3 algorithm and use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample is successfully executed and output is verified.

Exp: 9 **Date:**

Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

Aim:

To write a program to build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

Algorithm:

1. 1.initialize the weights and biases randomly.
2. For each training example:
    a. Perform a forward pass to compute the network output.
    b. Compute the error between the predicted and actual output.
    c. Perform a backward pass to propagate the error through the network.
    d. Update the weights and biases using gradient descent.
3. Repeat step 2 for all training examples and for multiple epochs until the error converges.
4. Test the trained network using new datasets.

Program:

```
import java.util.*;

public class NeuralNetwork {

   static class Neuron {

      double[] weights;

      double output;

      double delta;

      Neuron(int nInputs, Random rand) {

         weights = new double[nInputs + 1]; // +1 for bias

         for (int i = 0; i < weights.length; i++) {

            weights[i] = rand.nextDouble();

         }

      }

   }

   List<List<Neuron>> network;

   int nInputs, nHidden, nOutputs;
```

```
Random rand = new Random(1);

public NeuralNetwork(int nInputs, int nHidden, int nOutputs) {
    this.nInputs = nInputs;
    this.nHidden = nHidden;
    this.nOutputs = nOutputs;
    initializeNetwork();
}
private void initializeNetwork() {
    network = new ArrayList<>();
    // Hidden layer
    List<Neuron> hiddenLayer = new ArrayList<>();
    for (int i = 0; i < nHidden; i++) {
        hiddenLayer.add(new Neuron(nInputs, rand));
    }
    network.add(hiddenLayer);
    // Output layer
    List<Neuron> outputLayer = new ArrayList<>();
    for (int i = 0; i < nOutputs; i++) {
        outputLayer.add(new Neuron(nHidden, rand));
    }
    network.add(outputLayer);
}
private double activate(double[] weights, double[] inputs) {
    double activation = weights[weights.length - 1]; // bias
    for (int i = 0; i < weights.length - 1; i++) {
        activation += weights[i] * inputs[i];
    }
    return activation;
}
```

```
private double transfer(double activation) {
    return 1.0 / (1.0 + Math.exp(-activation));
}
private double transferDerivative(double output) {
    return output * (1.0 - output);
}
public double[] forwardPropagate(double[] inputs) {
    double[] currInputs = inputs;
    for (List<Neuron> layer : network) {
        double[] newInputs = new double[layer.size()];
        for (int i = 0; i < layer.size(); i++) {
            Neuron neuron = layer.get(i);
            double activation = activate(neuron.weights, currInputs);
            neuron.output = transfer(activation);
            newInputs[i] = neuron.output;
        }
        currInputs = newInputs;
    }
    return currInputs;
}
public void backwardPropagateError(double[] expected) {
    for (int i = network.size() - 1; i >= 0; i--) {
        List<Neuron> layer = network.get(i);
        List<Neuron> nextLayer = (i == network.size() - 1) ? null : network.get(i + 1);
        for (int j = 0; j < layer.size(); j++) {
            Neuron neuron = layer.get(j);
            double error = 0;
            if (nextLayer == null) {
                error = expected[j] - neuron.output;
            } else {
```

```
                for (Neuron nextNeuron : nextLayer) {
                    error += nextNeuron.weights[j] * nextNeuron.delta;
                }
            }
            neuron.delta = error * transferDerivative(neuron.output);
        }
    }
}
public void updateWeights(double[] row, double learningRate) {
    for (int i = 0; i < network.size(); i++) {
        double[] inputs = (i == 0) ? Arrays.copyOfRange(row, 0, row.length - 1)
                : getOutputs(network.get(i - 1));
        List<Neuron> layer = network.get(i);
        for (Neuron neuron : layer) {
            for (int j = 0; j < inputs.length; j++) {
                neuron.weights[j] += learningRate * neuron.delta * inputs[j];
            }
            neuron.weights[neuron.weights.length - 1] += learningRate * neuron.delta; // bias
        }
    }
}
private double[] getOutputs(List<Neuron> layer) {
    double[] outputs = new double[layer.size()];
    for (int i = 0; i < layer.size(); i++) {
        outputs[i] = layer.get(i).output;
    }
    return outputs;
}
public void trainNetwork(List<double[]> train, double learningRate, int nEpoch) {
    for (int epoch = 0; epoch < nEpoch; epoch++) {
```

```
        double sumError = 0;
        for (double[] row : train) {
            double[] outputs = forwardPropagate(row);
            double[] expected = new double[nOutputs];
            int targetIndex = (int) row[row.length - 1];
            expected[targetIndex] = 1.0;
            for (int i = 0; i < expected.length; i++) {
                sumError += Math.pow(expected[i] - outputs[i], 2);
            }
            backwardPropagateError(expected);
            updateWeights(row, learningRate);
        }
        System.out.printf(">epoch=%d, lrate=%.3f, error=%.3f%n", epoch, learningRate, sumError);
    }
}
public void printNetwork() {
    for (int i = 0; i < network.size(); i++) {
        System.out.println("Layer " + i + ":");
        for (Neuron neuron : network.get(i)) {
            System.out.println(Arrays.toString(neuron.weights));
        }
    }
}
public static void main(String[] args) {
    List<double[]> dataset = Arrays.asList(
            new double[]{2.7810836, 2.550537003, 0},
            new double[]{1.465489372, 2.362125076, 0},
            new double[]{3.396561688, 4.400293529, 0},
            new double[]{1.38807019, 1.850220317, 0},
            new double[]{3.06407232, 3.005305973, 0},
```

```
            new double[]{7.627531214, 2.759262235, 1},
            new double[]{5.332441248, 2.088626775, 1},
            new double[]{6.922596716, 1.77106367, 1},
            new double[]{8.675418651, -0.242068655, 1},
            new double[]{7.673756466, 3.508563011, 1}
        );
        int nInputs = dataset.get(0).length - 1;
        int nOutputs = 2;
        NeuralNetwork nn = new NeuralNetwork(nInputs, 2, nOutputs);
        nn.trainNetwork(dataset, 0.5, 20);
        nn.printNetwork();
    }
}
```

Output:

```
>epoch=0, lrate=0.500, error=7.838
>epoch=1, lrate=0.500, error=7.159
```

```
>epoch=2, lrate=0.500, error=6.471
>epoch=3, lrate=0.500, error=5.732
>epoch=4, lrate=0.500, error=5.288
>epoch=5, lrate=0.500, error=5.215
>epoch=6, lrate=0.500, error=5.225
>epoch=7, lrate=0.500, error=5.211
>epoch=8, lrate=0.500, error=5.156
>epoch=9, lrate=0.500, error=5.031
>epoch=10, lrate=0.500, error=4.751
>epoch=11, lrate=0.500, error=4.285
>epoch=12, lrate=0.500, error=3.826
>epoch=13, lrate=0.500, error=3.376
>epoch=14, lrate=0.500, error=2.941
>epoch=15, lrate=0.500, error=2.540
>epoch=16, lrate=0.500, error=2.185
>epoch=17, lrate=0.500, error=1.879
>epoch=18, lrate=0.500, error=1.622
>epoch=19, lrate=0.500, error=1.407
Layer 0:
[0.9262177067996287, -1.1164686687938716, -0.34094832867702884]
[-1.0668246752269814, 1.465791774597034, 0.16128808516454077]
Layer 1:
[-1.2764467651756222, 1.4933252823609302, -0.05295401461156517]
[1.6183133405363004, -1.4549787343824592, -0.1790150491397184]

=== Code Execution Successful ===
```

Result:

To write a program to build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets is successfully executed and output is verified.

Exp: 10 **Date:**

Write a program to implement the naïve Bayesian classifier for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets.

Aim:

To write a program to implement the naïve Bayesian classifier for a sample training data set stored as a .CSV file and compute the accuracy of the classifier, considering few test data sets.

Algorithm:

1. Load the training dataset from the .CSV file.
2. Calculate prior probabilities for each class.=
3. For each attribute, compute likelihood probabilities given each class.
4. For a new sample, compute the posterior probability for each class using Bayes' theorem.
5. Assign the class with the highest posterior probability to the new sample.
6. Test the classifier with a set of test data and compute accuracy.

Program:

```
import java.util.*;

public class Main {

   static class Summary {

      double mean, stdev;

      Summary(double m, double s) { mean = m; stdev = s; }

   }

   static List<double[]> dataset = Arrays.asList(

      new double[]{1, 1, 1, 1, 10}, new double[]{1, 1, 1, 2, 5}, new double[]{2, 1, 1, 1, 10},

      new double[]{3, 2, 1, 1, 10}, new double[]{3, 3, 2, 1, 5}, new double[]{3, 3, 2, 2, 10},

      new double[]{2, 3, 2, 2, 10}, new double[]{1, 2, 1, 1, 5}, new double[]{1, 3, 2, 1, 10},

      new double[]{3, 2, 2, 1, 10}, new double[]{1, 2, 2, 2, 10}, new double[]{2, 2, 1, 2, 10},

      new double[]{2, 1, 2, 1, 10}, new double[]{3, 2, 1, 2, 5}

   );

   public static void main(String[] args) {

      Collections.shuffle(dataset, new Random(42));
```

```
    int trainSize = (int) (dataset.size() * 0.75);

    List<double[]> train = dataset.subList(0, trainSize);
    List<double[]> test = dataset.subList(trainSize, dataset.size());
    Map<Double, List<Summary>> summaries = summarizeByClass(train);
    List<Double> predictions = new ArrayList<>();
    for (double[] row : test) predictions.add(predict(summaries, row));
    List<Double> actual = new ArrayList<>();
    for (double[] row : test) actual.add(row[row.length - 1]);
    System.out.println("Actual: " + actual);
    System.out.println("Predicted: " + predictions);
    System.out.printf("Accuracy: %.2f%%\n", getAccuracy(test, predictions));
}

static Map<Double, List<Summary>> summarizeByClass(List<double[]> data) {
    Map<Double, List<double[]>> separated = new HashMap<>();
    for (double[] row : data) {
        double label = row[row.length - 1];
        separated.computeIfAbsent(label, k -> new ArrayList<>()).add(row);
    }
    Map<Double, List<Summary>> summaries = new HashMap<>();
    for (var entry : separated.entrySet()) summaries.put(entry.getKey(), summarize(entry.getValue()));
    return summaries;
}
static List<Summary> summarize(List<double[]> data) {
    int len = data.get(0).length - 1;
    List<Summary> result = new ArrayList<>();
    for (int i = 0; i < len; i++) {
        List<Double> values = new ArrayList<>();
        for (double[] row : data) values.add(row[i]);
```

```
            result.add(new Summary(mean(values), stdev(values)));
        }
        return result;
    }
    static double mean(List<Double> list) {
        double sum = 0;
        for (double x : list) sum += x;
        return sum / list.size();
    }

    static double stdev(List<Double> list) {
        double avg = mean(list), sum = 0;
        for (double x : list) sum += Math.pow(x - avg, 2);
        return Math.sqrt(sum / (list.size() - 1));
    }
    static double calculateProbability(double x, double mean, double stdev) {
        if (stdev == 0) return x == mean ? 1 : 0;
        double exp = Math.exp(-Math.pow(x - mean, 2) / (2 * stdev * stdev));
        return (1 / (Math.sqrt(2 * Math.PI) * stdev)) * exp;
    }
    static double predict(Map<Double, List<Summary>> summaries, double[] row) {
        double bestLabel = -1, bestProb = -1;
        for (var entry : summaries.entrySet()) {
            double prob = 1.0;
            for (int i = 0; i < entry.getValue().size(); i++) {
                Summary s = entry.getValue().get(i);
                prob *= calculateProbability(row[i], s.mean, s.stdev);
            }
            if (prob > bestProb) {
                bestProb = prob;
```

```
            bestLabel = entry.getKey();
        }
    }

    return bestLabel;
  }
  static double getAccuracy(List<double[]> testSet, List<Double> predictions) {
    int correct = 0;
    for (int i = 0; i < testSet.size(); i++)
      if (testSet.get(i)[testSet.get(i).length - 1] == predictions.get(i)) correct++;
    return (double) correct / testSet.size() * 100;
  }
}
```

Output:

```
Actual: [10.0, 10.0, 5.0, 10.0]
Predicted: [5.0, 10.0, 10.0, 10.0]
Accuracy: 50.00%


...Program finished with exit code 0
Press ENTER to exit console.
```

Result:

Thus a program to implement the naïve Bayesian classifier for a sample training data set stored as a .CSV file and compute the accuracy of the classifier, considering few test data sets.is successfully executed and output is verified.